

비동기 처리

비동기 처리의 원리

타이머 함수



타이머 함수

setTimeout

setInterval

자바스크립트에는 특수한 역할을 하는 함수가 두 개 존재한다.



setTimeout

setInterval

즉시 실행되는 일반 함수 호출과는 달리,
위 두 함수를 사용하면 함수 호출을 예약할 수 있다.



setTimeout

```
console.log('3초 후에 메시지가 출력됩니다.');
```

```
setTimeout(() => {  
  console.log('3초가 지났습니다!');  
}, 3000);
```

‘setTimeout’ 함수를 사용하면 특정 시간 후에 콜백 함수가 호출되도록 예약할 수 있다.

두 번째 인수로 시간을 전달하며, 단위는 밀리초(ms)이다.



setTimeout

```
setTimeout((name) => console.log(`Hello, ${name}!`), 2000, "Elice") // 2초 후에 'Hello, Elice!' 출력
```

세 번째 이후의 인수에는 콜백 함수에 전달될 인수를 전달할 수 있다.

```
const timeoutId = setTimeout(() => console.log(`Hello!`), 2000);  
  
console.log(timeoutId); // 1  
clearTimeout(timeoutId);
```

‘setTimeout’ 함수는 숫자 형태의 timeout id를 반환한다. (브라우저 환경 기준)

‘clearTimeout’ 함수에 이 id를 인수로 넘겨 호출하면 해당 timeout을 제거할 수 있다.

```
let count = 0;

const intervalId = setInterval(() => {
  count++;
  console.log(`Interval 실행: ${count}초 경과`);
}, 1000); // 1초마다 실행
```

```
'Interval 실행: 1초 경과'
'Interval 실행: 2초 경과'
'Interval 실행: 3초 경과'
'Interval 실행: 4초 경과'
'Interval 실행: 5초 경과'
'Interval 실행: 6초 경과'
'Interval 실행: 7초 경과'
'Interval 실행: 8초 경과'
...
```

‘setInterval’ 함수를 사용하면 특정 시간마다 콜백 함수가 호출되도록 예약할 수 있다.

두 번째 인수로 시간을 전달하며, 단위는 밀리초(ms)이다.


```
let count = 0;

const intervalId = setInterval(() => {
  count++;
  console.log(`Interval 실행: ${count}초 경과`);

  if (count === 5) {
    clearInterval(intervalId); // 5초 후에 반복 중지
    console.log('Interval 중지');
  }
}, 1000); // 1초마다 실행
```

```
'Interval 실행: 1초 경과'
'Interval 실행: 2초 경과'
'Interval 실행: 3초 경과'
'Interval 실행: 4초 경과'
'Interval 실행: 5초 경과'
'Interval 중지'
```

‘setInterval’ 함수는 숫자 형태의 interval id를 반환한다. (브라우저 환경 기준)

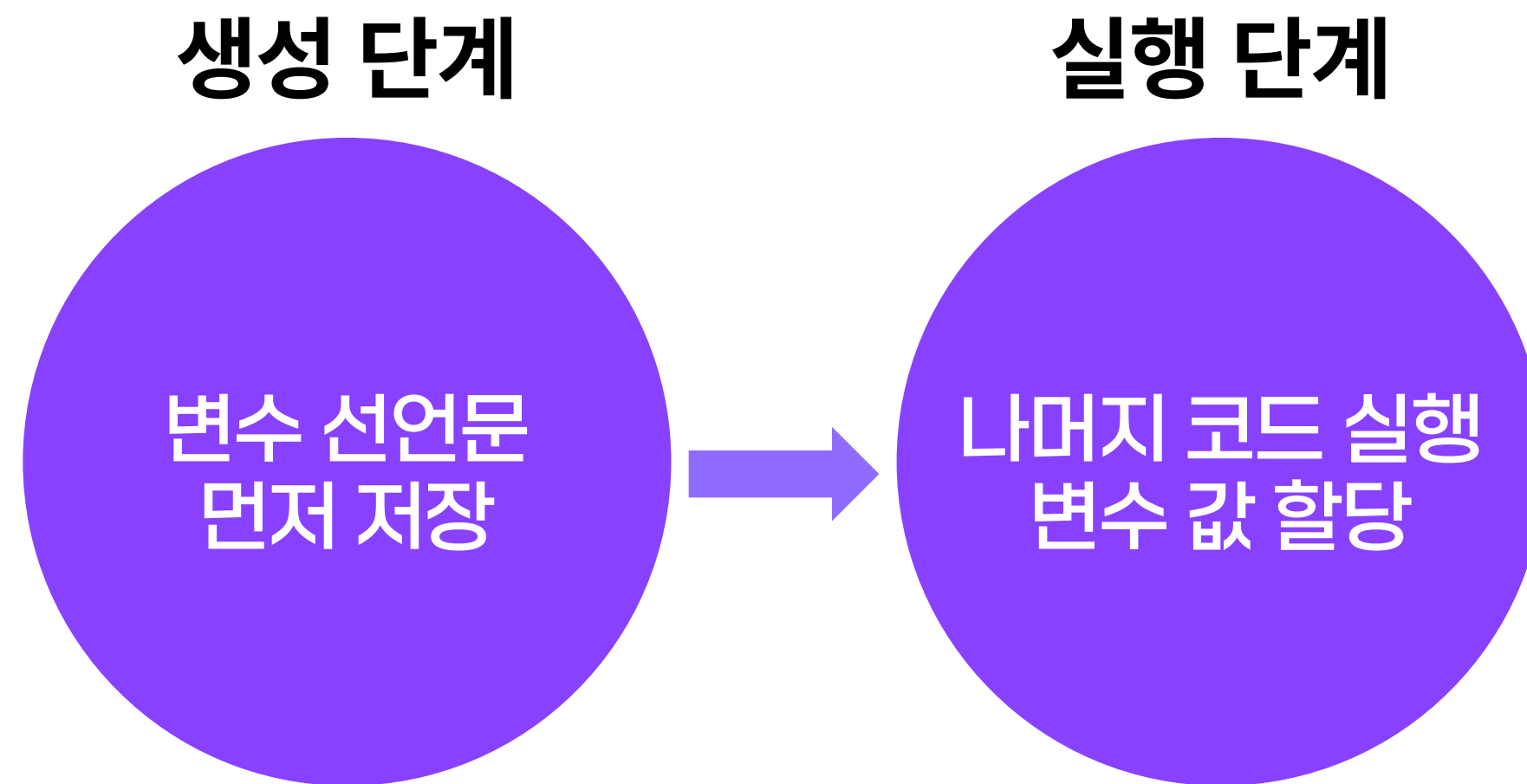
‘clearInterval’ 함수에 이 id를 인수로 넘겨 호출하면 해당 interval을 제거할 수 있다.

실행 컨텍스트



작성한 자바스크립트 코드는 자바스크립트 엔진을 통해 파싱되고 실행된다.

가장 많이 사용하는 Chrome 브라우저의 경우 V8 엔진을 사용한다.



자바스크립트 엔진은 코드 실행 전 실행 컨텍스트*라는 것을 생성한다.
실행 컨텍스트는 위의 두 단계를 통해 생성 및 사용된다.



실행 컨텍스트

```
const x = 10;

function func1() {
  const x = 20;

  func2();
}

function func2() {
  console.log(x);
}

func1();
func2();
```

전역 실행 컨텍스트 생성

그리고 실행 컨텍스트는 각 스코프가 실행되기 직전에 생성된다.



실행 컨텍스트

```
const x = 10;

function func1() {
  const x = 20;

  func2();
}

function func2() {
  console.log(x);
}

func1();
func2();
```

func1 실행 컨텍스트 생성



그리고 실행 컨텍스트는 각 스코프가 실행되기 직전에 생성된다.



실행 컨텍스트

```
const x = 10;

function func1() {
  const x = 20;

  func2();
}

function func2() {
  console.log(x);
}

func1();
func2();
```

func2 실행 컨텍스트 생성

그리고 실행 컨텍스트는 각 스코프가 실행되기 직전에 생성된다.



실행 컨텍스트 스택

```
const global = 10;

function outerFunc() {
  const global = 20;
  const local1 = 30;

  function innerFunc() {
    const local1 = 40;
    const local2 = 50;

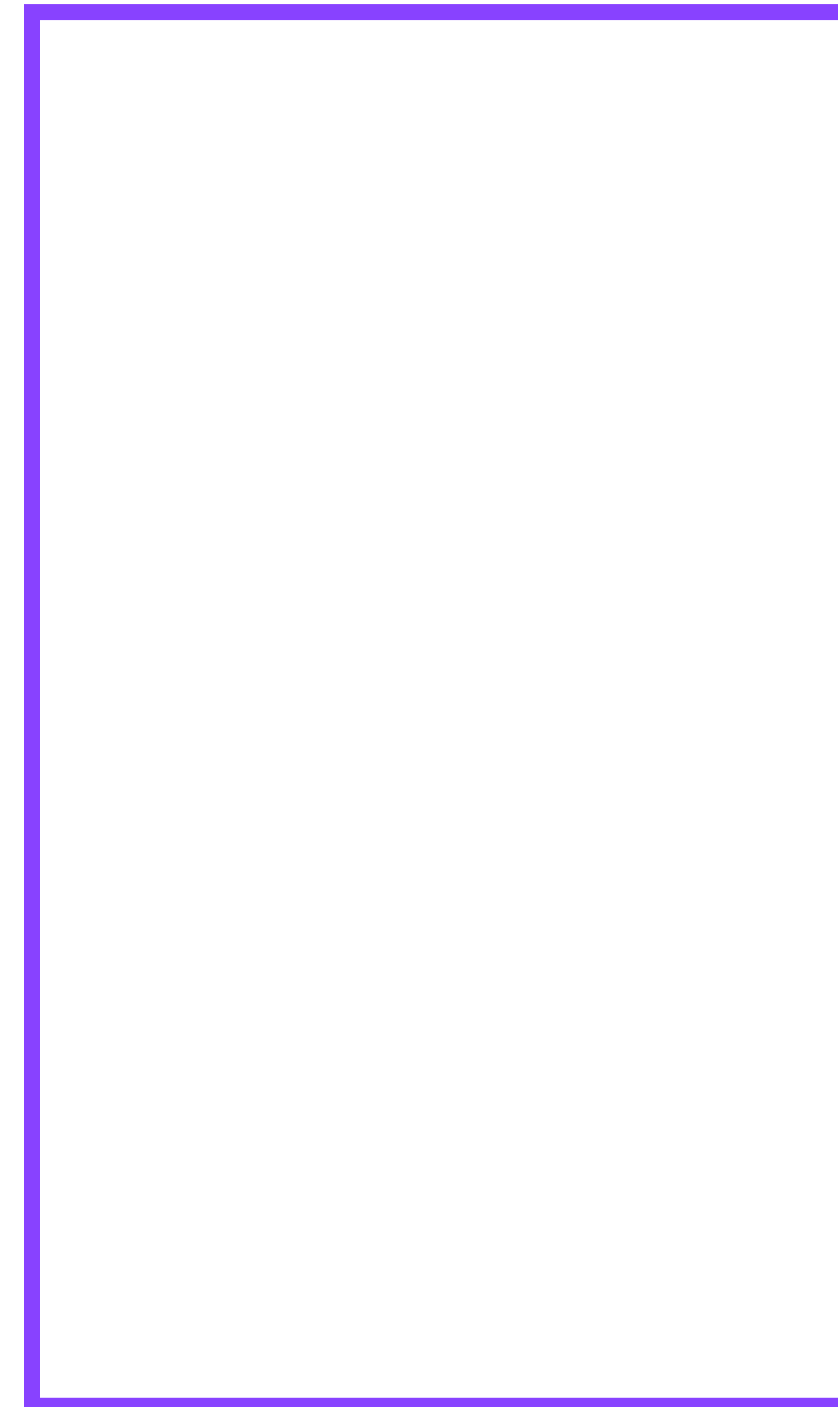
    console.log("inner function");
    console.log(global);
    console.log(local1);
    console.log(local2);
  }

  console.log("outer function");
  console.log(global);
  console.log(local1);

  innerFunc();
}

console.log("global");
console.log(global);

outerFunc();
```



생성되는 실행 컨텍스트는 모두 실행 컨텍스트 스택(콜 스택)에서 관리된다.

스택(Stack): 후입선출(LIFO, Last In First Out) 원칙에 따라 데이터를 저장하고 제거하는 선형 자료구조



실행 컨텍스트 스택

```
const global = 10;

function outerFunc() {
  const global = 20;
  const local1 = 30;

  function innerFunc() {
    const local1 = 40;
    const local2 = 50;

    console.log("inner function");
    console.log(global);
    console.log(local1);
    console.log(local2);
  }

  console.log("outer function");
  console.log(global);
  console.log(local1);

  innerFunc();
}

console.log("global");
console.log(global);

outerFunc();
```

전역 실행 컨텍스트

생성되는 실행 컨텍스트는 모두 실행 컨텍스트 스택(콜 스택)에서 관리된다.

스택(Stack): 후입선출(LIFO, Last In First Out) 원칙에 따라 데이터를 저장하고 제거하는 선형 자료구조



실행 컨텍스트 스택

```
const global = 10;

function outerFunc() {
  const global = 20;
  const local1 = 30;

  function innerFunc() {
    const local1 = 40;
    const local2 = 50;

    console.log("inner function");
    console.log(global);
    console.log(local1);
    console.log(local2);
  }

  console.log("outer function");
  console.log(global);
  console.log(local1);

  innerFunc();
}

console.log("global");
console.log(global);

outerFunc();
```

함수 outerFunc
실행 컨텍스트

전역 실행 컨텍스트

생성되는 실행 컨텍스트는 모두 실행 컨텍스트 스택(콜 스택)에서 관리된다.

스택(Stack): 후입선출(LIFO, Last In First Out) 원칙에 따라 데이터를 저장하고 제거하는 선형 자료구조



실행 컨텍스트 스택

```
const global = 10;

function outerFunc() {
  const global = 20;
  const local1 = 30;

  function innerFunc() {
    const local1 = 40;
    const local2 = 50;

    console.log("inner function");
    console.log(global);
    console.log(local1);
    console.log(local2);
  }

  console.log("outer function");
  console.log(global);
  console.log(local1);

  innerFunc();
}

console.log("global");
console.log(global);

outerFunc();
```

함수 innerFunc
실행 컨텍스트

함수 outerFunc
실행 컨텍스트

전역 실행 컨텍스트

생성되는 실행 컨텍스트는 모두 실행 컨텍스트 스택(콜 스택)에서 관리된다.

스택(Stack): 후입선출(LIFO, Last In First Out) 원칙에 따라 데이터를 저장하고 제거하는 선형 자료구조



실행 컨텍스트 스택

```
const global = 10;

function outerFunc() {
  const global = 20;
  const local1 = 30;

  function innerFunc() {
    const local1 = 40;
    const local2 = 50;

    console.log("inner function");
    console.log(global);
    console.log(local1);
    console.log(local2);
  }

  console.log("outer function");
  console.log(global);
  console.log(local1);

  innerFunc();
}

console.log("global");
console.log(global);

outerFunc();
```

함수 outerFunc
실행 컨텍스트

전역 실행 컨텍스트

생성되는 실행 컨텍스트는 모두 실행 컨텍스트 스택(콜 스택)에서 관리된다.

스택(Stack): 후입선출(LIFO, Last In First Out) 원칙에 따라 데이터를 저장하고 제거하는 선형 자료구조



실행 컨텍스트 스택

```
const global = 10;

function outerFunc() {
  const global = 20;
  const local1 = 30;

  function innerFunc() {
    const local1 = 40;
    const local2 = 50;

    console.log("inner function");
    console.log(global);
    console.log(local1);
    console.log(local2);
  }

  console.log("outer function");
  console.log(global);
  console.log(local1);

  innerFunc();
}

console.log("global");
console.log(global);

outerFunc();
```

전역 실행 컨텍스트

생성되는 실행 컨텍스트는 모두 실행 컨텍스트 스택(콜 스택)에서 관리된다.

스택(Stack): 후입선출(LIFO, Last In First Out) 원칙에 따라 데이터를 저장하고 제거하는 선형 자료구조



실행 컨텍스트 스택

```
const global = 10;

function outerFunc() {
  const global = 20;
  const local1 = 30;

  function innerFunc() {
    const local1 = 40;
    const local2 = 50;

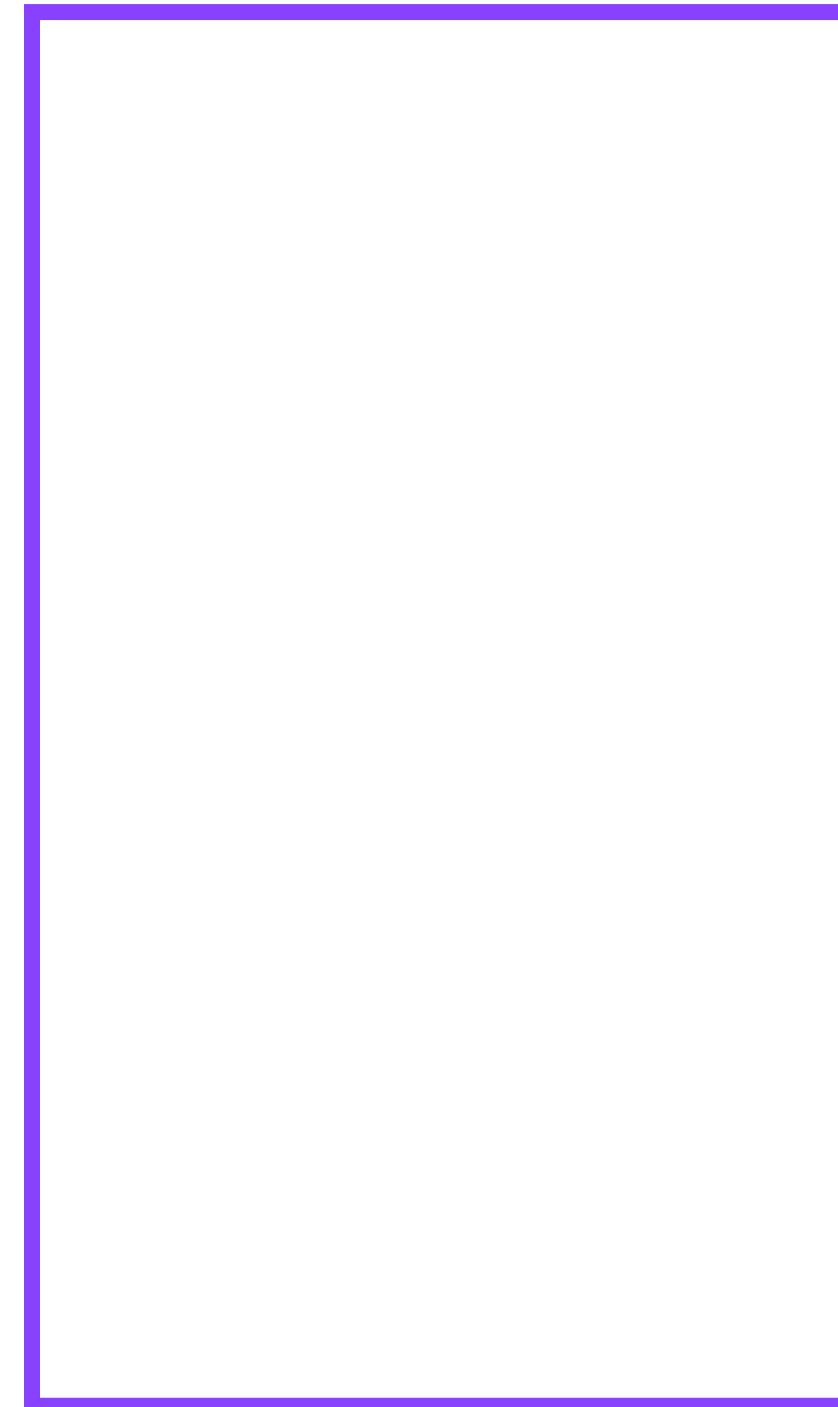
    console.log("inner function");
    console.log(global);
    console.log(local1);
    console.log(local2);
  }

  console.log("outer function");
  console.log(global);
  console.log(local1);

  innerFunc();
}

console.log("global");
console.log(global);

outerFunc();
```

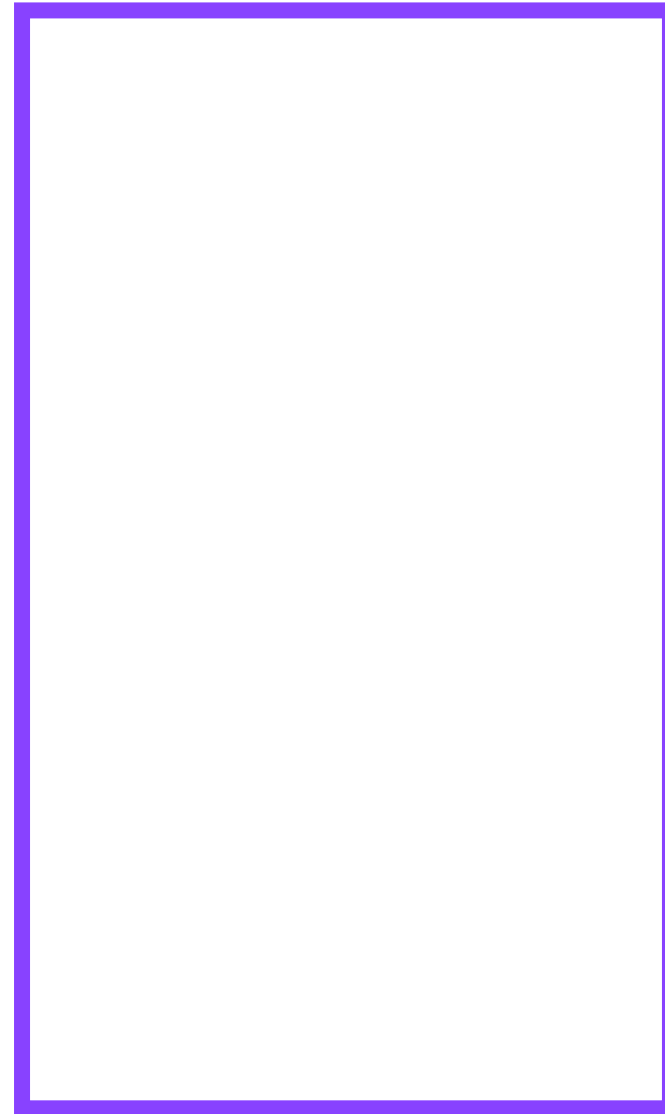


생성되는 실행 컨텍스트는 모두 실행 컨텍스트 스택(콜 스택)에서 관리된다.

스택(Stack): 후입선출(LIFO, Last In First Out) 원칙에 따라 데이터를 저장하고 제거하는 선형 자료구조



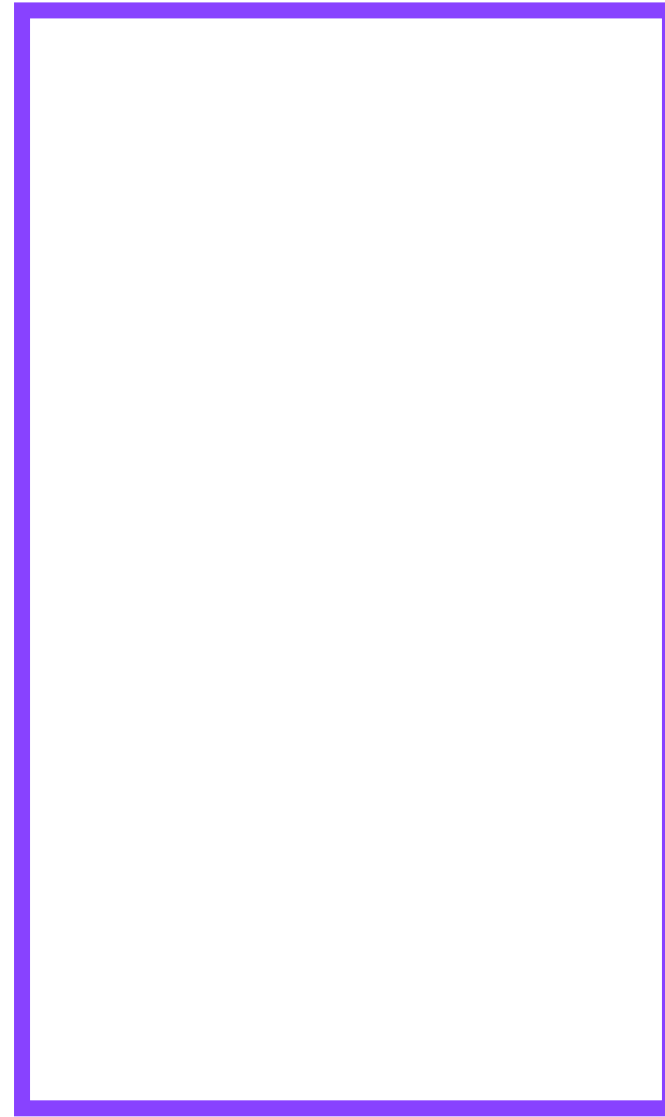
싱글 스레드 (Single Thread)



자바스크립트는 오직 하나의 실행 컨텍스트 스택(콜 스택)을 가진다.
즉, 한 번에 하나의 작업만 수행할 수 있다.



싱글 스레드 (Single Thread)



= 자바스크립트는 싱글 스레드 기반 언어이다.

동기와 비동기



싱글 스레드의 문제점

```
const fs = require('fs');

console.log('파일 읽기 시작');

// 동기적인 파일 읽기
const data = fs.readFileSync('largeFile.txt', 'utf8');

console.log('파일 읽기 완료');
console.log(data);

console.log('다른 작업 실행');
```

싱글 스레드에서는 한 번에 하나의 작업만 수행하므로 작업 순서가 보장된다는 장점이 있지만, 앞선 작업이 종료되기 전까지 이후 작업을 시작하지 못한다는 문제가 있다.



싱글 스레드의 문제점

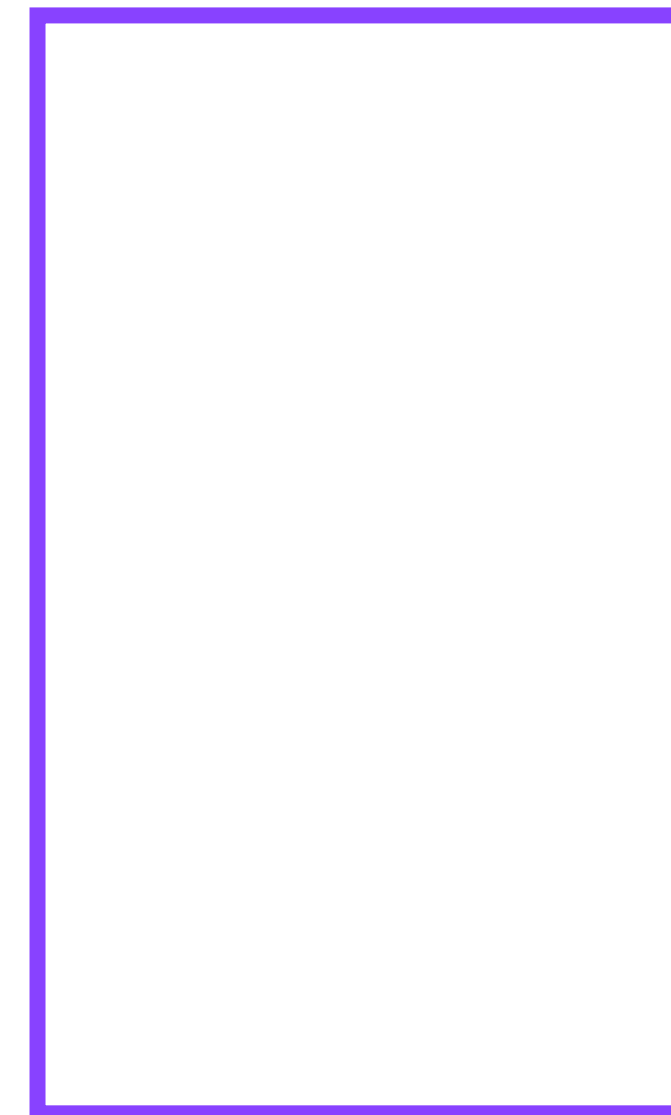
```
const fs = require('fs');

console.log('파일 읽기 시작');

// 동기적인 파일 읽기
const data = fs.readFileSync('largeFile.txt', 'utf8');

console.log('파일 읽기 완료');
console.log(data);

console.log('다른 작업 실행');
```



앞선 작업이 종료되기 전까지 이후 작업을 시작하지 못한다는 문제가 있다.

= 블로킹(Blocking)이 발생한다.



싱글 스레드의 문제점

```
const fs = require('fs');  
  
console.log('파일 읽기 시작');  
  
// 동기적인 파일 읽기  
const data = fs.readFileSync('largeFile.txt', 'utf8');  
  
console.log('파일 읽기 완료');  
console.log(data);  
  
console.log('다른 작업 실행');
```



```
fs.readFileSync('largeFile.txt', 'utf8');
```

앞선 작업이 종료되기 전까지 이후 작업을 시작하지 못한다는 문제가 있다.

= 블로킹(Blocking)이 발생한다.

동기 (Synchronous) 처리 방식

```
const fs = require('fs');  
  
console.log('파일 읽기 시작');  
  
// 동기적인 파일 읽기  
const data = fs.readFileSync('largeFile.txt', 'utf8');  
  
console.log('파일 읽기 완료');  
console.log(data);  
  
console.log('다른 작업 실행');
```



```
fs.readFileSync('largeFile.txt', 'utf8');
```

동기 (Synchronous) 처리 방식

작업이 순차적으로 실행되며, 현재 작업이 완료될 때까지 다음 작업이 대기하는 방식

비동기 (Asynchronous) 처리 방식

```
const fs = require('fs');

console.log('파일 읽기 시작');

// 비동기적인 파일 읽기
fs.readFile('largeFile.txt', 'utf8', (err, data) => {
  if (err) throw err;
  console.log('파일 읽기 완료');
  console.log(data);
});

console.log('다른 작업 실행');
```

비동기 (Asynchronous) 처리 방식

작업이 시작된 후 완료 여부와 상관없이 다음 작업을 즉시 실행하며,
작업 완료 시 콜백 함수나 프로미스 등을 통해 결과를 처리하는 방식

비동기 (Asynchronous) 처리 방식

```
const fs = require('fs');

console.log('파일 읽기 시작');

// 비동기적인 파일 읽기
fs.readFile('largeFile.txt', 'utf8', (err, data) => {
  if (err) throw err;
  console.log('파일 읽기 완료');
  console.log(data);
});

console.log('다른 작업 실행');
```

```
fs.readFile
('largeFile.txt', 'utf8',
() => { ... });
```

비동기 (Asynchronous) 처리 방식

작업이 시작된 후 완료 여부와 상관없이 다음 작업을 즉시 실행하며,
작업 완료 시 콜백 함수나 프로미스 등을 통해 결과를 처리하는 방식

비동기 (Asynchronous) 처리 방식

```
const fs = require('fs');

console.log('파일 읽기 시작');

// 비동기적인 파일 읽기
fs.readFile('largeFile.txt', 'utf8', (err, data) => {
  if (err) throw err;
  console.log('파일 읽기 완료');
  console.log(data);
});

console.log('다른 작업 실행');
```

```
console.log('다른  
작업 실행');
```

비동기 (Asynchronous) 처리 방식

작업이 시작된 후 완료 여부와 상관없이 다음 작업을 즉시 실행하며,
작업 완료 시 콜백 함수나 프로미스 등을 통해 결과를 처리하는 방식

비동기 (Asynchronous) 처리 방식

```
const fs = require('fs');

console.log('파일 읽기 시작');

// 비동기적인 파일 읽기
fs.readFile('largeFile.txt', 'utf8', (err, data) => {
  if (err) throw err;
  console.log('파일 읽기 완료');
  console.log(data);
});

console.log('다른 작업 실행');
```

파일 로드 완료 후

(err, data) => { ... }

비동기 (Asynchronous) 처리 방식

작업이 시작된 후 완료 여부와 상관없이 다음 작업을 즉시 실행하며,
작업 완료 시 콜백 함수나 프로미스 등을 통해 결과를 처리하는 방식

비동기 (Asynchronous) 처리 방식 종류

타이머 함수

(setTimeout / setInterval)

Ajax

(XMLHttpRequest, Fetch API 등)

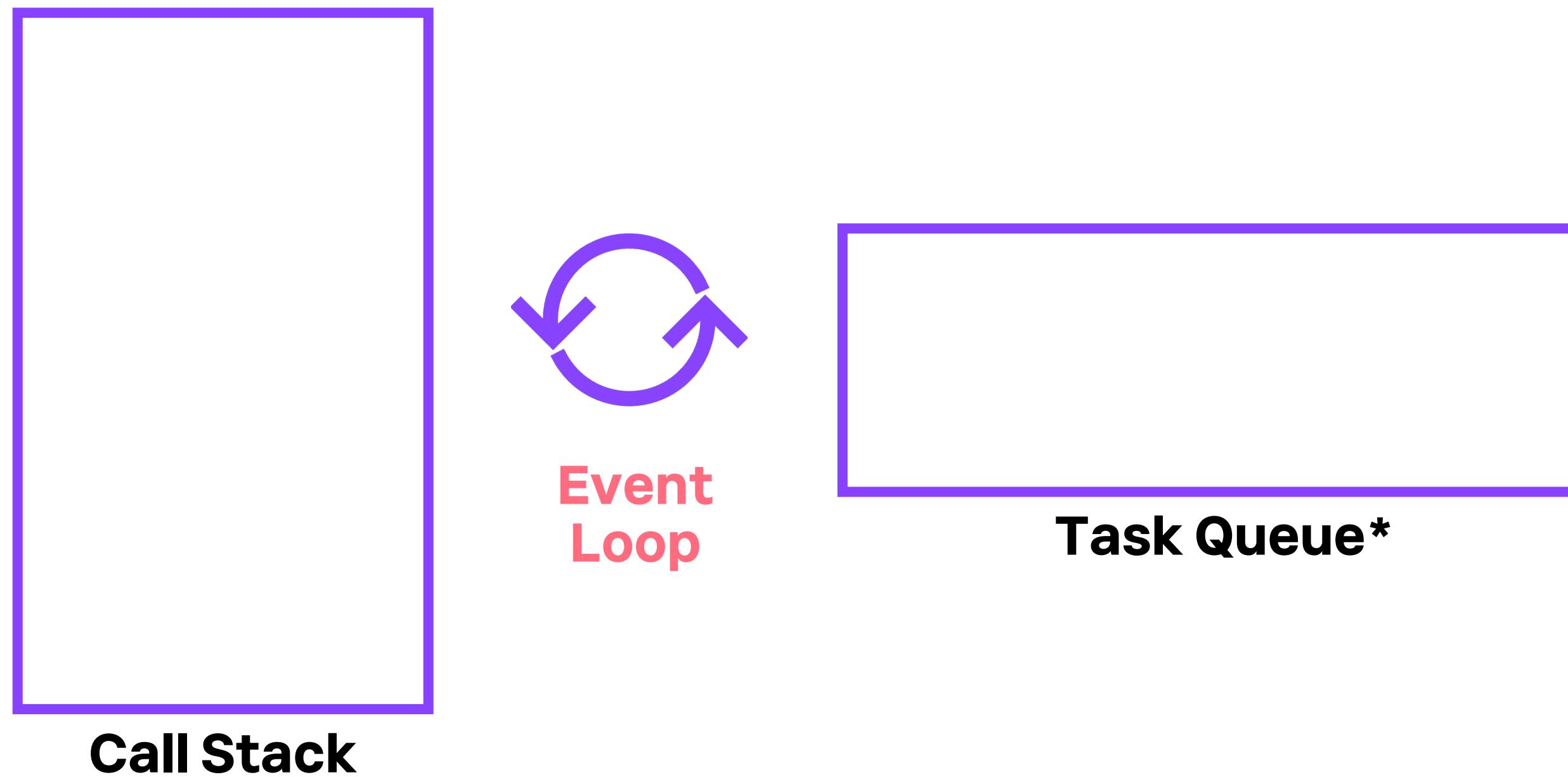
이벤트 핸들러

(addEventListener)

비동기 처리 방식으로 동작하는 자바스크립트 기능은
위 3가지이다.

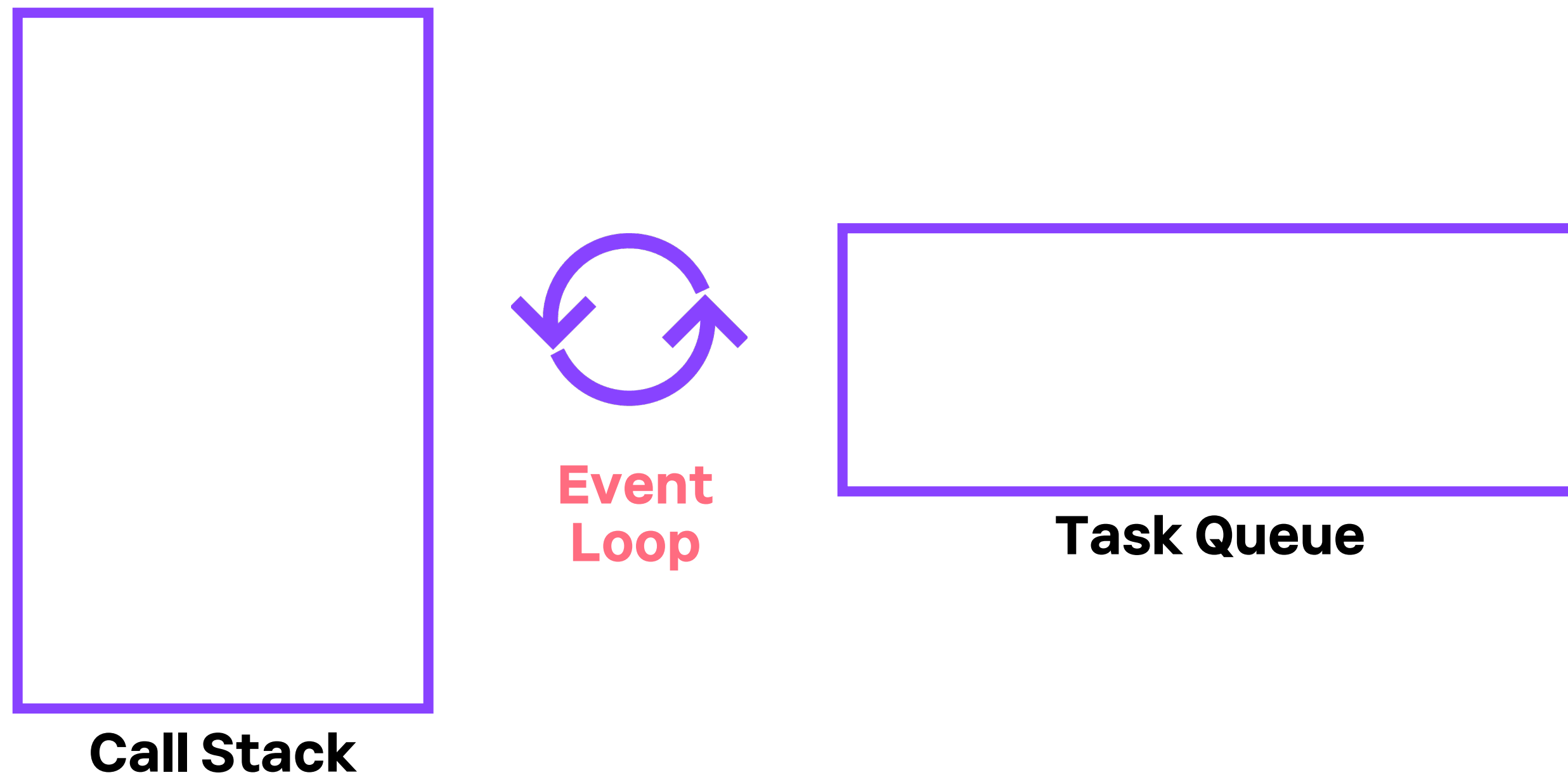
비동기 동작 원리

비동기 (Asynchronous) 동작 원리



브라우저는 이벤트 루프 (Event Loop)와 태스크 큐 (Task Queue)를 이용해 비동기를 처리한다.

비동기 (Asynchronous) 동작 원리



태스크 큐* (Task Queue): 이벤트 핸들러 및 비동기 함수의 콜백 함수가 잠시 저장되는 큐 영역

이벤트 루프 (Event Loop): 콜 스택이 비어 있고, 태스크 큐에 함수가 존재하면 이를 콜 스택으로 이동시킴



비동기 (Asynchronous) 동작 원리

```
console.log('1. 스크립트 시작');

setTimeout(() => {
  console.log('3. setTimeout 콜백 실행');
}, 0);

console.log('2. 스크립트 끝');
```

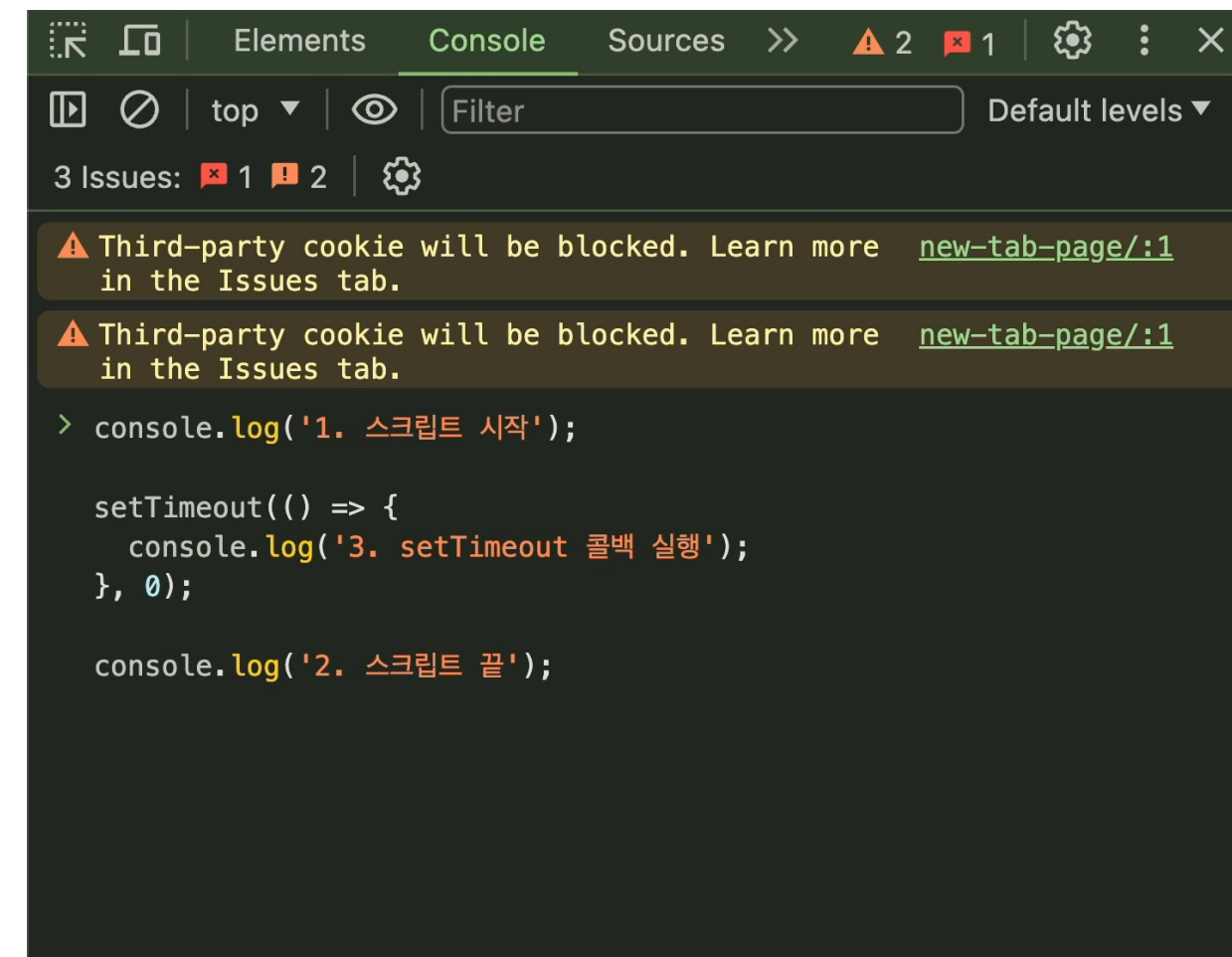
위 코드를 동기 처리 방식으로 실행하면,
1 → 3 → 2 순서로 출력되는 것이 정상이다.

비동기 (Asynchronous) 동작 원리

```
console.log('1. 스크립트 시작');

setTimeout(() => {
  console.log('3. setTimeout 콜백 실행');
}, 0);

console.log('2. 스크립트 끝');
```



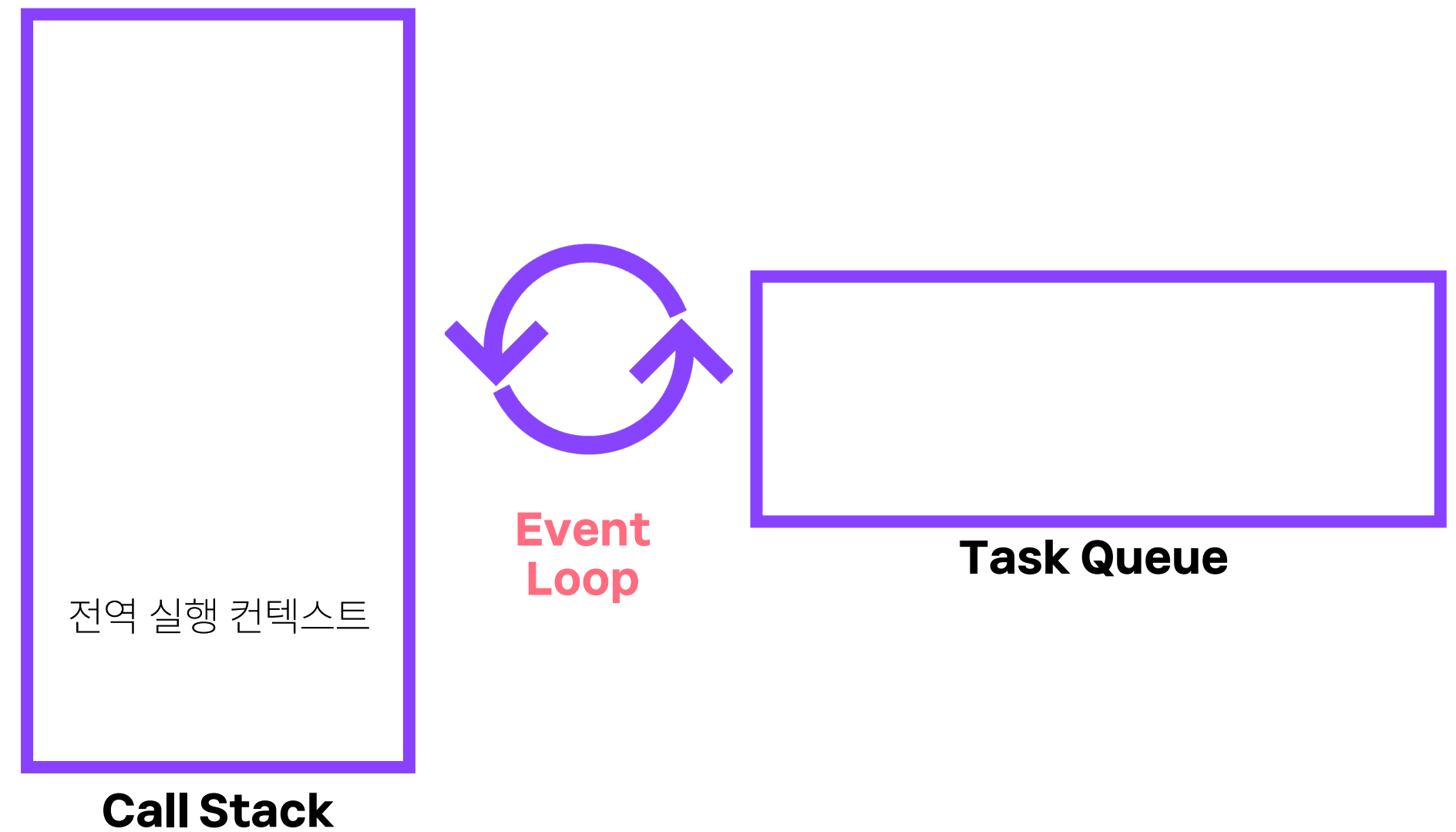
그러나 실제로는 1 → 2 → 3 순으로 출력된다.
비동기의 동작 원리를 이해해 보면서 이유를 확인해보자.

비동기 (Asynchronous) 동작 원리

```
console.log('1. 스크립트 시작');

setTimeout(() => {
  console.log('3. setTimeout 콜백 실행');
}, 0);

console.log('2. 스크립트 끝');
```

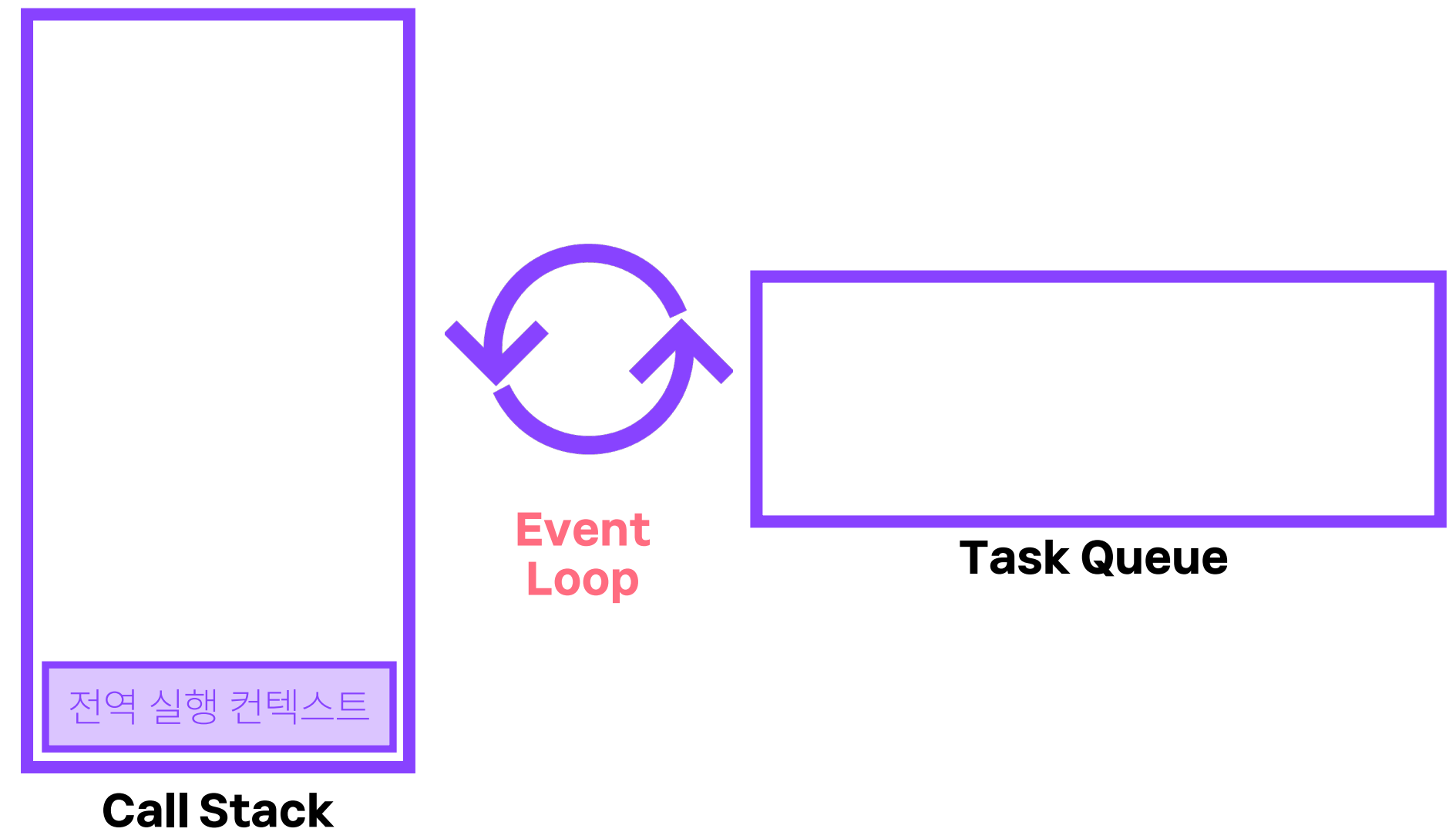


비동기 (Asynchronous) 동작 원리

```
console.log('1. 스크립트 시작');

setTimeout(() => {
  console.log('3. setTimeout 콜백 실행');
}, 0);

console.log('2. 스크립트 끝');
```

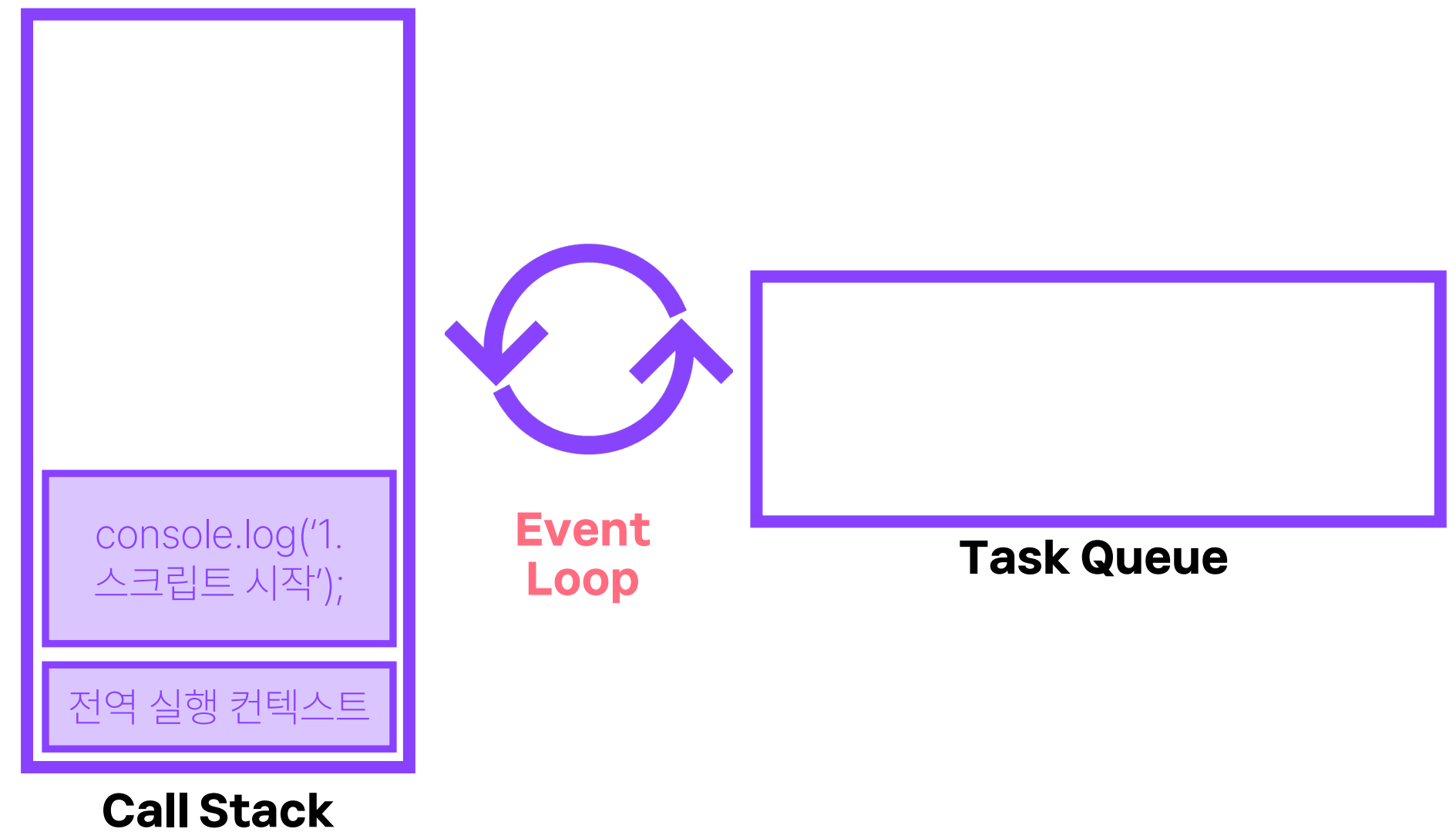


비동기 (Asynchronous) 동작 원리

```
console.log('1. 스크립트 시작');

setTimeout(() => {
  console.log('3. setTimeout 콜백 실행');
}, 0);

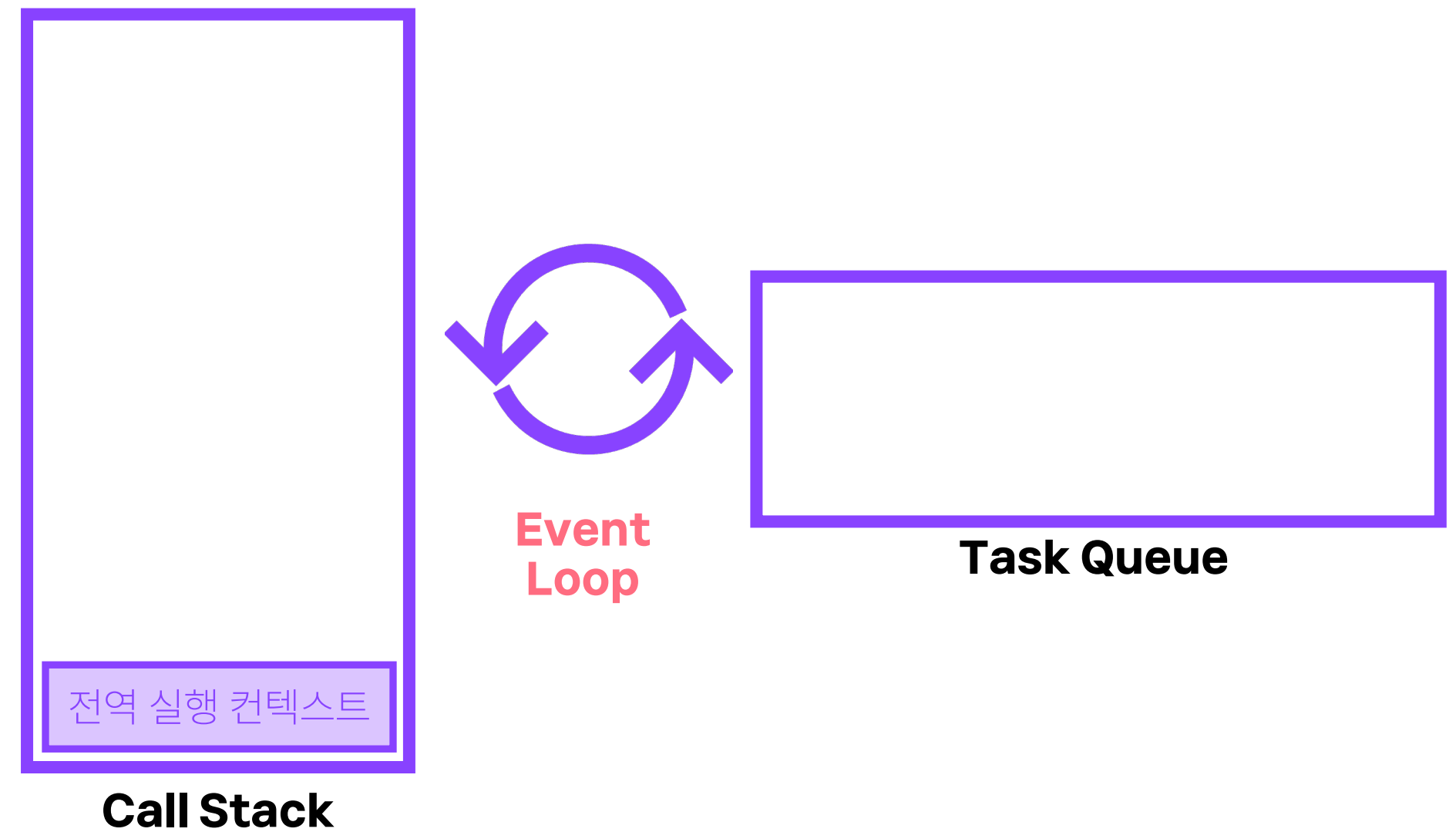
console.log('2. 스크립트 끝');
```



비동기 (Asynchronous) 동작 원리

```
console.log('1. 스크립트 시작');  
  
setTimeout(() => {  
  console.log('3. setTimeout 콜백 실행');  
}, 0);  
  
console.log('2. 스크립트 끝');
```

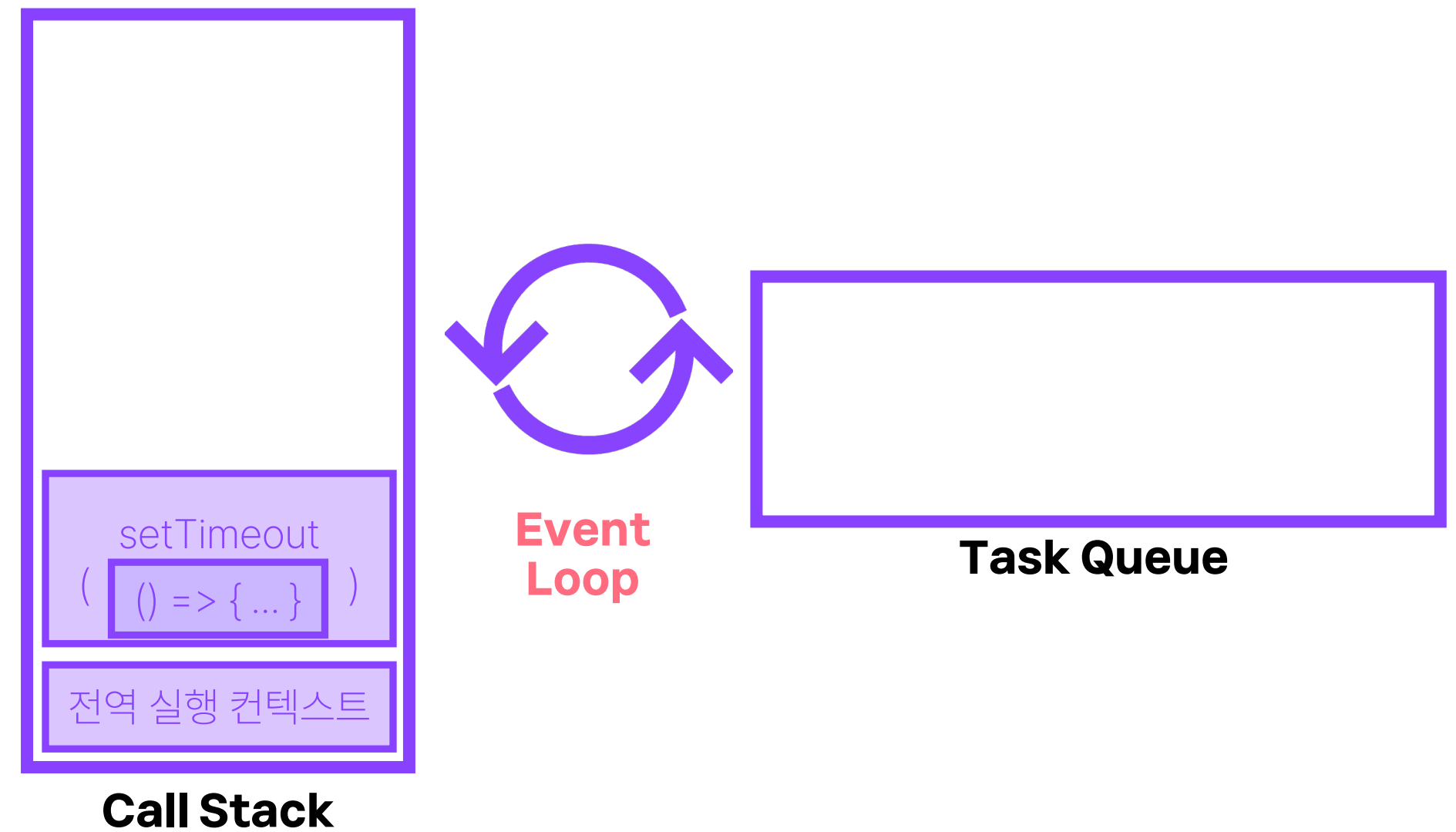
1. 스크립트 시작



비동기 (Asynchronous) 동작 원리

```
console.log('1. 스크립트 시작');  
  
setTimeout(() => {  
  console.log('3. setTimeout 콜백 실행');  
}, 0);  
  
console.log('2. 스크립트 끝');
```

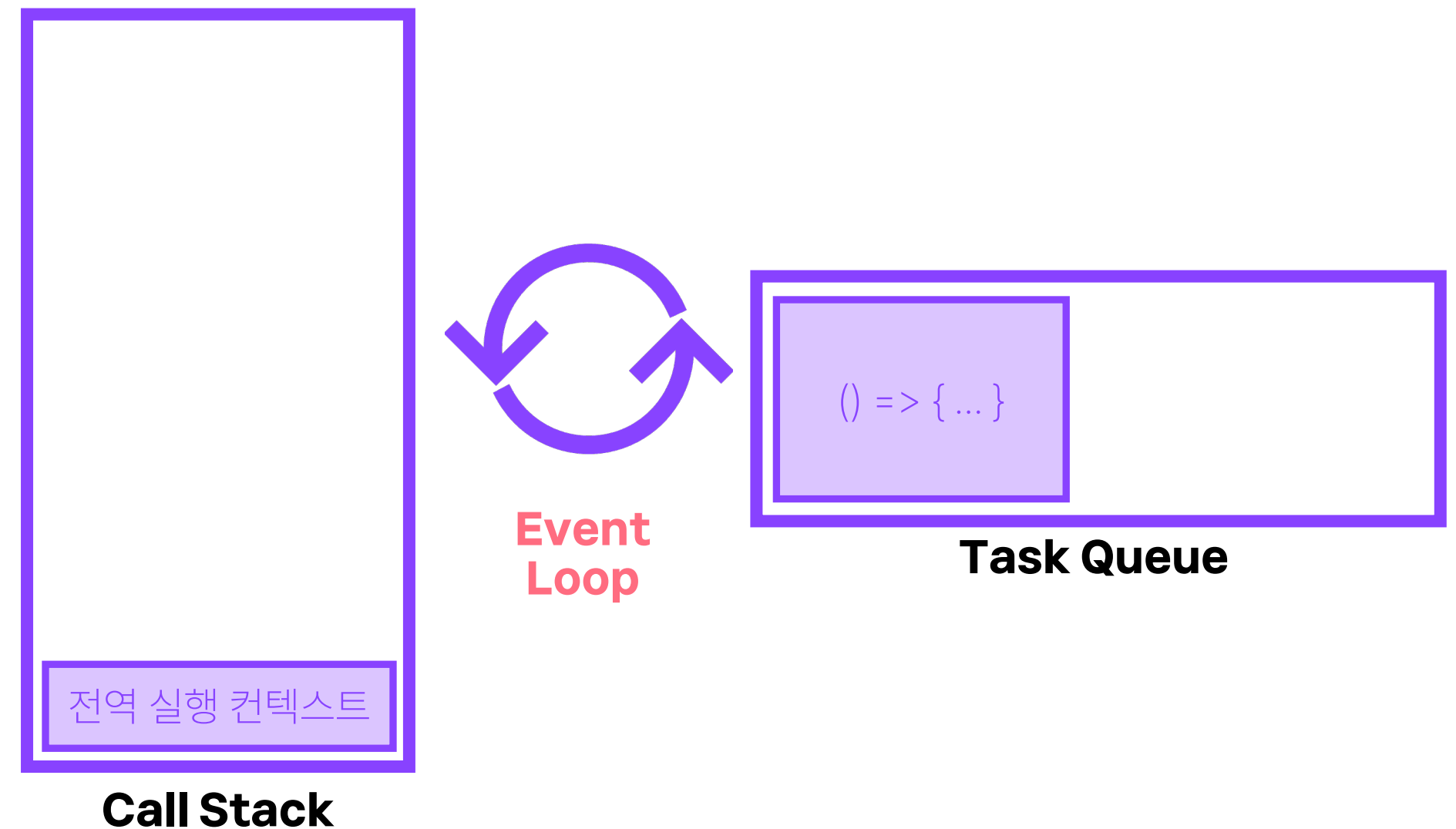
1. 스크립트 시작



비동기 (Asynchronous) 동작 원리

```
console.log('1. 스크립트 시작');  
  
setTimeout(() => {  
  console.log('3. setTimeout 콜백 실행');  
}, 0);  
  
console.log('2. 스크립트 끝');
```

1. 스크립트 시작



이벤트 루프가 비동기 함수의 콜백 함수를 태스크 큐에 저장한다.

(setTimeout의 경우, 지정된 밀리초가 지난 후에 태스크 큐에 저장된다.)

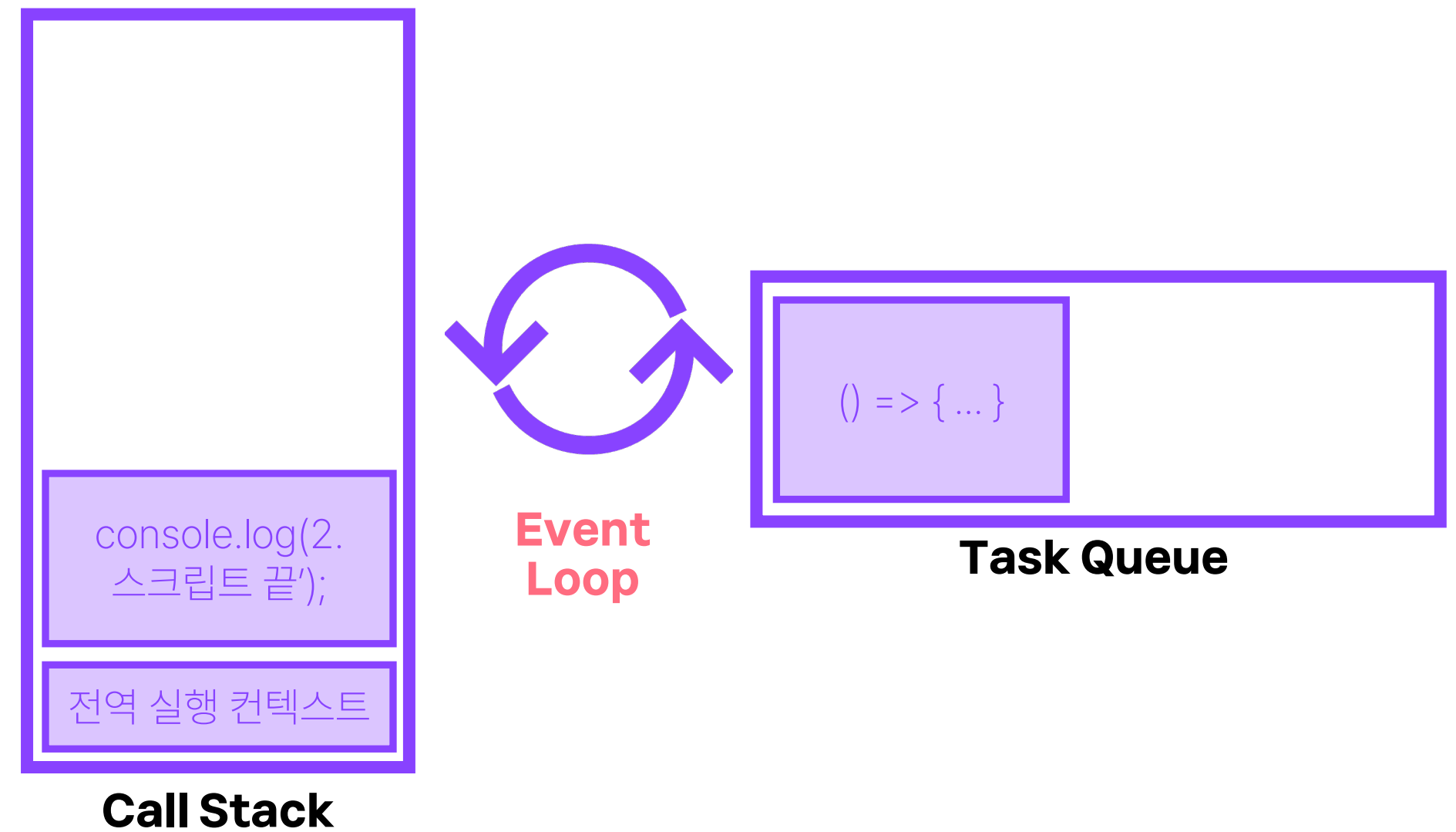
비동기 (Asynchronous) 동작 원리

```
console.log('1. 스크립트 시작');

setTimeout(() => {
  console.log('3. setTimeout 콜백 실행');
}, 0);

console.log('2. 스크립트 끝');
```

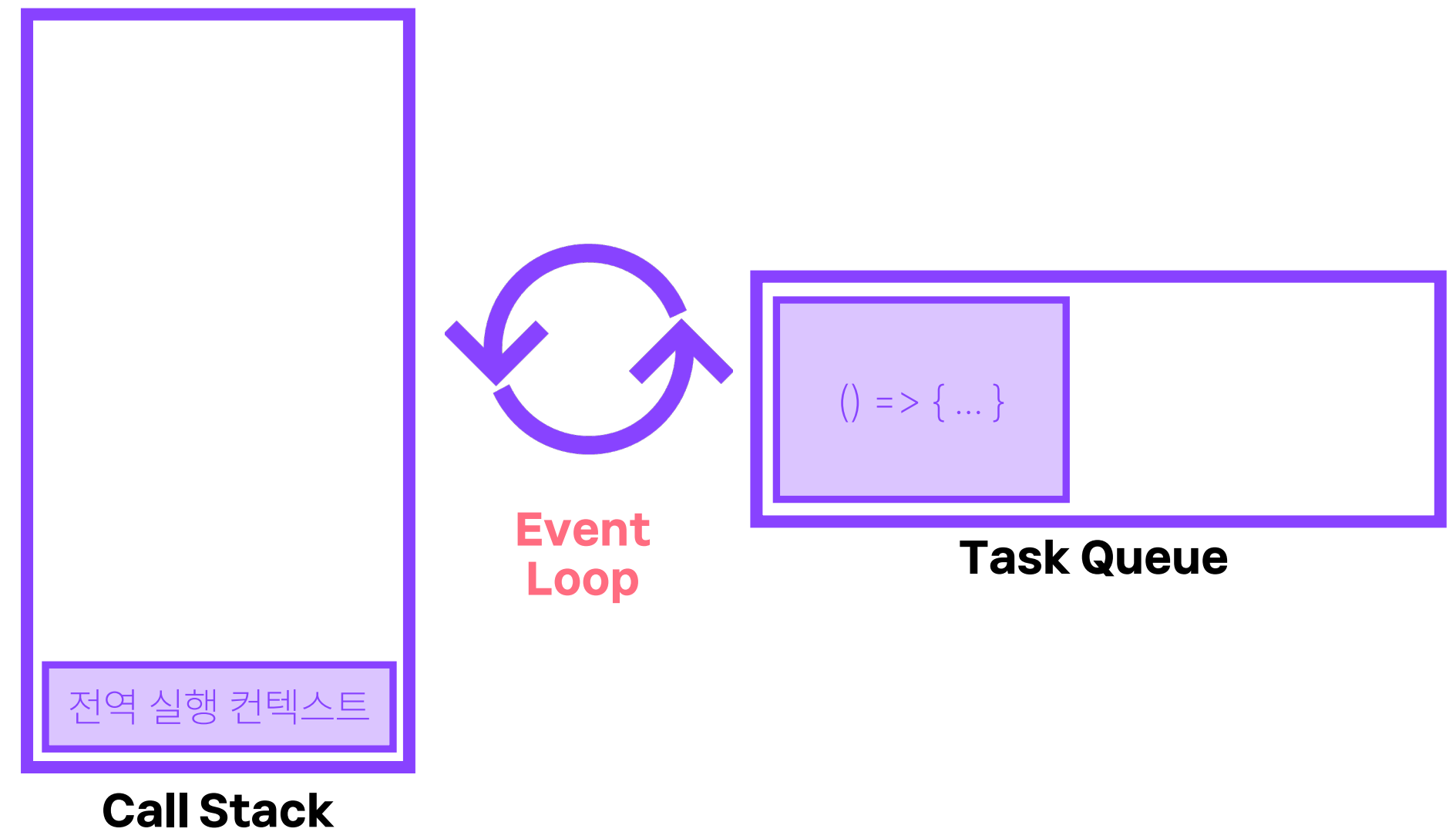
1. 스크립트 시작



비동기 (Asynchronous) 동작 원리

```
console.log('1. 스크립트 시작');  
  
setTimeout(() => {  
  console.log('3. setTimeout 콜백 실행');  
}, 0);  
  
console.log('2. 스크립트 끝');
```

1. 스크립트 시작
2. 스크립트 끝



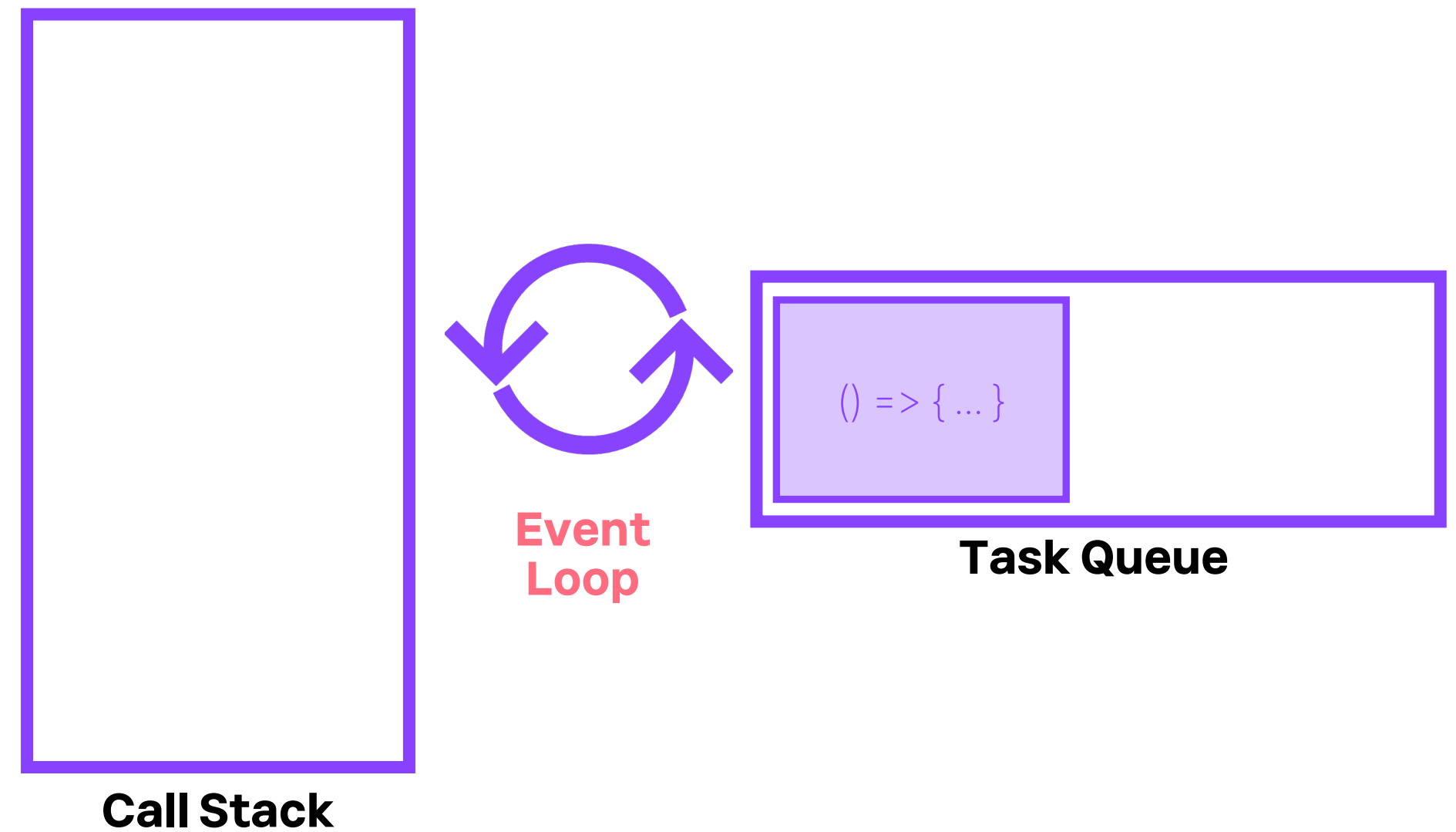
비동기 (Asynchronous) 동작 원리

```
console.log('1. 스크립트 시작');

setTimeout(() => {
  console.log('3. setTimeout 콜백 실행');
}, 0);

console.log('2. 스크립트 끝');
```

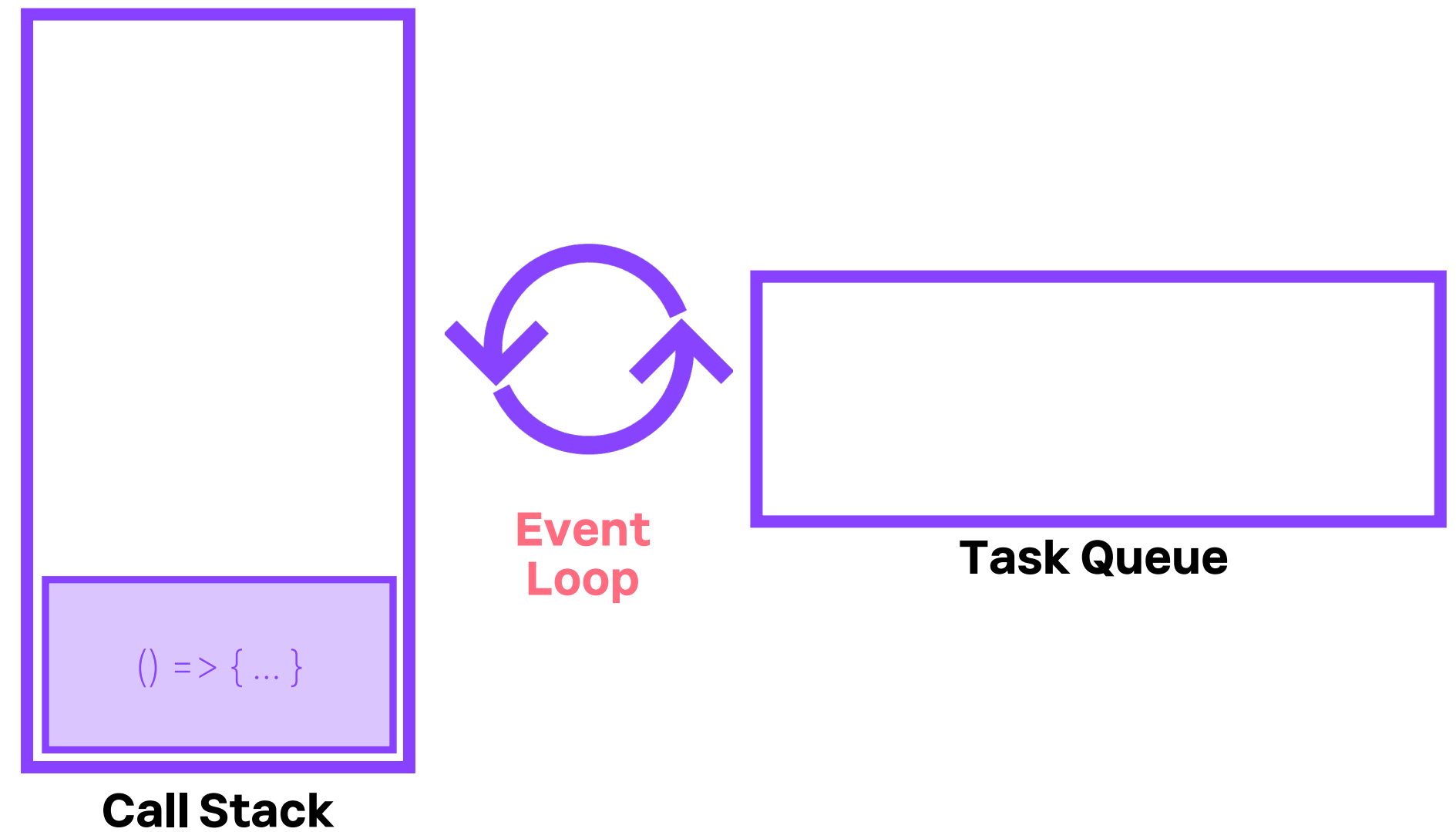
```
1. 스크립트 시작
2. 스크립트 끝
```



비동기 (Asynchronous) 동작 원리

```
console.log('1. 스크립트 시작');  
  
setTimeout(() => {  
  console.log('3. setTimeout 콜백 실행');  
}, 0);  
  
console.log('2. 스크립트 끝');
```

```
1. 스크립트 시작  
2. 스크립트 끝
```



콜 스택이 비면, 이벤트 루프는 태스크 큐에 있는 작업을 콜 스택으로 이동시킨다.

비동기 (Asynchronous) 동작 원리

```
console.log('1. 스크립트 시작');  
  
setTimeout(() => {  
  console.log('3. setTimeout 콜백 실행');  
}, 0);  
  
console.log('2. 스크립트 끝');
```

1. 스크립트 시작
2. 스크립트 끝
3. setTimeout 콜백 실행

